

Step 1 — A Minimal Server With One Tool — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real, long-lived server would handle this). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The pieces, and where each one lives](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The pieces, and where each one lives

Basic Version

This step only proves one thing: your code can start an MCP server, and something (the Inspector, or a tiny script) can connect to it and call its one tool. Nothing else yet — no database, no second tool.

You need three small pieces: a config loader for `LOG_LEVEL` (reuse Doc01's, don't rewrite it), a `FastMCP` server object, and one function decorated as a tool that stores text in a plain Python list.

Things to use: - `from mcp.server.fastmcp import FastMCP` — the server class. - `mcp = FastMCP("notes-server")` — build it once, give it a name. - `@mcp.tool()` — the decorator that turns a plain function into something a client can call. - `mcp.run(transport="stdio")` — actually starts the server listening. Nothing works until this line runs.

Don't overthink storage yet — one list, appended to, nothing else.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The real shape you're aiming for is one small file, `server.py`, with one decorated function:

```
@mcp.tool()
def add_note(text: str) -> str:
```

`FastMCP` reads the function's type hints (`text: str`) the same way `Doc06` taught you a Pydantic argument model gets built for a plain API tool call — you don't write a separate schema by hand, the decorator builds it from the signature. The docstring becomes the tool's description, exactly the same "the description is really a prompt" idea from `Doc06` — write a real one-sentence description, not `"""Adds a note."""`.

The in-memory list needs to live somewhere the function can reach on every call — a module-level list declared once, near the top of the file, is the simplest correct place for it in a script this small.

`mcp.run(transport="stdio")` needs to sit behind `if __name__ == "__main__":`, same reason you'd guard any script's entry point — so importing `server.py` from elsewhere (which Step 4's tests might eventually do) doesn't accidentally start the server as a side effect.

Sketch the file's shape — imports, the list, the decorated function, the `if __name__` guard — before writing the bodies.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

`server.py` proves the connection works, but only once you've actually opened the Inspector or run a client script — nothing checks the server is even syntactically able to start before that. A real project separates two different questions, the same way Doc06's "readiness vs. liveness" idea (see Project 1's Step 1 hints, if you built that project already) applies here too: "does the process start at all" and "does a client calling it get a sane answer."

Think about what happens on the stdio transport specifically. The client launches your `server.py` as a subprocess and talks to it entirely over stdin/stdout — nothing else is allowed to touch stdout. If any code above `mcp.run()` (an import that prints a warning, a leftover debug `print()`) writes to stdout before the protocol handshake starts, the client sees garbage instead of the expected handshake bytes, and the connection fails in a way that looks nothing like your actual bug.

The fix, once and for all, for this entire project: route every bit of your own output through `logging_setup.py`'s logger, configured to write to stderr (the standard default for Python's `logging` module) — never a bare `print()`, anywhere in `server.py`, `db.py`, or `client.py`. This isn't just style here the way it might be in a terminal app — on stdio, a stray `print()` is a protocol-breaking bug, not just noisy output.

Also worth doing once, by hand: run `python server.py` directly (not through the Inspector) and confirm it doesn't crash and doesn't print anything to your terminal — it should just sit there, silently waiting on stdin, until you `Ctrl+C` it. That silence is exactly correct behavior for a stdio server with no client attached yet.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get the server responding correctly once a client is already attached and working. Advanced explains *why* stdout discipline isn't optional here the way it might be elsewhere — a single misplaced `print()` breaks the protocol itself, not just the log output — and gives you a way to sanity-check the server starts cleanly before you ever open the Inspector.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make an empty list called notes

create the server, name it "notes-server"

define add_note(text):
    add text to notes
    return a confirmation string

run the server over stdio
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The same plan, closer to real structure:

```
config.py:
    load_config() -> Config          (from Doc01, reused as-is; just LOG_LEVEL th

logging_setup.py:
    get_logger(name) -> Logger      (from Doc01, reused as-is)

server.py:
    _notes: list[str] = []          (module-level, in-memory only)

    mcp = FastMCP("notes-server")

    @mcp.tool()
    def add_note(text: str) -> str:
        _notes.append(text)
        return f"Note added. There are now {len(_notes)} note(s)."
```

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Write this yourself, run `mcp dev server.py`, and call `add_note` through the Inspector's UI before looking at Advanced.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the missing piece from Hint 1 — logging routed correctly, and a real tool description, instead of a one-word docstring:

```
from mcp.server.fastmcp import FastMCP
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

_notes: list[str] = []

mcp = FastMCP("notes-server")

@mcp.tool()
def add_note(text: str) -> str:
    """Add a new note to the local notes store. Use this whenever the user
    wants to save, remember, or jot down a piece of text. Returns a short
    confirmation once the note is stored."""
    _notes.append(text)
    logger.info("Note added (in-memory, count=%d)", len(_notes))
    return f"Note added. There are now {len(_notes)} note(s)."
```

```
if __name__ == "__main__":
    logger.info("Starting notes-server over stdio")
    mcp.run(transport="stdio")
```

Try running `python server.py` directly first — confirm it prints nothing to your terminal and just waits. Then run `mcp dev server.py` and call `add_note` twice in the same Inspector session, confirming the count climbs to 2. Restart the server (close and reopen the Inspector connection) and confirm the count resets to 0 — proving, on purpose, that this version really is only in-memory. That's exactly the gap Step 2 closes.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get a working tool. Advanced adds the logging discipline this whole project depends on (stderr only, never `print()`, because stdout is the protocol channel), a real tool description instead of a placeholder, and a way to confirm the server starts cleanly *before* you ever open a client against it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)